

uniApp Vue3 + Vite 完整模板（含路由+Pinia+请求拦截+tabBar）

说明：本模板为 uniApp Vue3 + Vite + setup 语法糖，整合了路由封装、Pinia 全局状态管理、tabBar 底部导航、请求拦截（token/加载/错误处理），开箱即用，复制所有文件即可新建项目开发。

一、项目完整结构

```
1  /pages
2    /index          # 首页（tabBar页面）
3      index.vue
4    /mine           # 我的（tabBar页面）
5      index.vue
6    /detail         # 详情页（普通页面）
7      index.vue
8  /components
9    /Hello.vue      # 示例组件
10 /store
11   /index.js        # Pinia 全局状态管理入口
12   /modules
13     /user.js       # 用户模块状态
14 /utils
15   /request.js      # 请求封装（含拦截器）
16   /router.js       # 路由封装
17 App.vue           # 应用入口
18 main.js           # 入口JS（注册Pinia、路由等）
19 pages.json         # 页面路由+tabBar配置
20 manifest.json      # 多端配置
21 uni.scss           # 全局样式变量
22 vite.config.js     # Vite配置
23 package.json       # 依赖+脚本命令
```

二、核心配置文件（直接复制）

1. package.json（依赖+运行/打包命令）

```
1  {
```

```

2   "name": "uni-app-v3-complete",
3   "version": "1.0.0",
4   "scripts": {
5     "dev:h5": "uni",
6     "dev:mp-weixin": "uni --mp-weixin",
7     "dev:app": "uni --app",
8     "build:h5": "uni build",
9     "build:mp-weixin": "uni build --mp-weixin",
10    "build:app": "uni build --app"
11  },
12  "dependencies": {
13    "@dcloudio/uni-app": "^3.0.0",
14    "@dcloudio/uni-components": "^3.0.0",
15    "@dcloudio/uni-helper-json": "^1.0.13",
16    "pinia": "^2.1.7",
17    "vue": "^3.3.4"
18  },
19  "devDependencies": {
20    "@dcloudio/vite-plugin-uni": "^3.0.0",
21    "vite": "~5.0.0"
22  }
23 }

```

2. vite.config.js (Vite基础配置)

```

1  import { defineConfig } from 'vite'
2  import uni from '@dcloudio/vite-plugin-uni'
3
4  export default defineConfig({
5    plugins: [uni()],
6    // 可选: 配置路径别名, 简化导入
7    resolve: {
8      alias: {
9        '@': '/',
10       '@utils': '/utils',
11       '@store': '/store',
12       '@components': '/components'
13     }
14   }
15 })

```

3. main.js (入口文件, 注册Pinia)

```

1  import { createSSRApp } from 'vue'
2  import App from './App.vue'
3  // 引入Pinia
4  import { createPinia } from 'pinia'
5
6  export function createApp() {
7    const app = createSSRApp(App)
8    // 注册Pinia到全局
9    const pinia = createPinia()
10   app.use(pinia)
11   return { app }
12 }

```

4. App.vue（应用入口，全局样式）

```

1  <script setup>
2  // 全局引入路由封装（可选，也可在页面单独引入）
3  import '@/utils/router.js'
4  </script>
5
6  <template>
7    <view id="app">
8      <router-view />
9    </view>
10 </template>
11
12 <style>
13 /* 全局样式：解决多端适配基础问题 */
14 page {
15   background-color: #f5f5f5;
16   font-size: 28rpx;
17   color: #333;
18 }
19 /* 清除默认边距 */
20 * {
21   margin: 0;
22   padding: 0;
23   box-sizing: border-box;
24 }
25 </style>

```

5. pages.json（路由+tabBar配置）

```
1  {
2    "pages": [
3      {
4        "path": "pages/index/index",
5        "style": {
6          "navigationBarTitleText": "首页",
7          "navigationBarBackgroundColor": "#409eff",
8          "navigationBarTextStyle": "white"
9        }
10     },
11     {
12       "path": "pages/mine/mine",
13       "style": {
14         "navigationBarTitleText": "我的",
15         "navigationBarBackgroundColor": "#409eff",
16         "navigationBarTextStyle": "white"
17       }
18     },
19     {
20       "path": "pages/detail/index",
21       "style": {
22         "navigationBarTitleText": "详情页",
23         "navigationBarBackgroundColor": "#409eff",
24         "navigationBarTextStyle": "white"
25       }
26     }
27   ],
28   "globalStyle": {
29     "navigationBarTextStyle": "white",
30     "backgroundTextStyle": "light",
31     "navigationBarBackgroundColor": "#409eff",
32     "backgroundColor": "#f5f5f5"
33   },
34   // tabBar 底部导航配置
35   "tabBar": {
36     "color": "#666",
37     "selectedColor": "#409eff",
38     "backgroundColor": "#fff",
39     "borderStyle": "black",
40     "list": [
41       {
42         "pagePath": "pages/index/index",
43         "text": "首页",
44         // 可替换为自己的图标 (本地/网络)
45         "iconPath": "static/home.png",
46         "selectedIconPath": "static/home-active.png"
47       }
48     ]
49   }
50 }
```

```
48     {
49         "pagePath": "pages/mine/mine",
50         "text": "我的",
51         "iconPath": "static/mine.png",
52         "selectedIconPath": "static/mine-active.png"
53     }
54 ]
55 },
56 "uniIdRouter": false
57 }
```

6. manifest.json（多端基础配置，可直接使用）

```
1  {
2      "name": "uniApp Vue3 完整模板",
3      "appid": "__UNI__XXXXXX", // 替换为自己的uniApp appid
4      "description": "uniApp Vue3 + Vite 完整模板，含路由、Pinia、请求拦截",
5      "versionName": "1.0.0",
6      "versionCode": 100,
7      "transformPx": true,
8      "uniStatistics": {
9          "enable": true
10     },
11     "app-plus": {
12         "usingComponents": true,
13         "splashscreen": {
14             "alwaysShowBeforeRender": true,
15             "waiting": true,
16             "autoclose": true,
17             "delay": 0
18         }
19     },
20     "mp-weixin": {
21         "appid": "", // 替换为自己的微信小程序appid
22         "setting": {
23             "urlCheck": false
24         },
25         "usingComponents": true
26     },
27     "h5": {
28         "usingComponents": true,
29         "router": {
30             "mode": "hash"
31         }
32     }
```

三、核心工具封装（路由+请求）

1. utils/router.js（路由封装，简化跳转）

```

1  /**
2   * uniApp Vue3 路由封装
3   * 简化跳转逻辑，统一处理参数传递
4   */
5  export const router = {
6    // 保留当前页，跳转到新页面（非tabBar页面）
7    push: (url, params = {}) => {
8      // 拼接参数（如： /pages/detail/index?id=1&name=test）
9      const query = Object.entries(params)
10        .map(([key, value]) => `${key}=${value}`)
11        .join('&')
12      const fullUrl = query ? `${url}?${query}` : url
13      uni.navigateTo({ url: fullUrl })
14    },
15
16    // 关闭当前页，跳转到新页面（非tabBar页面）
17    replace: (url, params = {}) => {
18      const query = Object.entries(params)
19        .map(([key, value]) => `${key}=${value}`)
20        .join('&')
21      const fullUrl = query ? `${url}?${query}` : url
22      uni.redirectTo({ url: fullUrl })
23    },
24
25    // 切换tabBar页面（只能跳转tabBar配置的页面）
26    switchTab: (url) => {
27      uni.switchTab({ url })
28    },
29
30    // 关闭所有页面，跳转到新页面（如登录页）
31    reLaunch: (url, params = {}) => {
32      const query = Object.entries(params)
33        .map(([key, value]) => `${key}=${value}`)
34        .join('&')
35      const fullUrl = query ? `${url}?${query}` : url
36      uni.reLaunch({ url: fullUrl })
37    },
38  }

```

```

39    // 返回上一页 (可指定返回页数)
40    back: (delta = 1) => {
41        uni.navigateBack({ delta })
42    }
43 }
44
45 // 全局挂载 (可选, 在页面中可直接使用 this.$router, 需在main.js注册)
46 // 若不全局挂载, 可在页面单独引入: import { router } from '@utils/router.js'
47 import { getCurrentInstance } from 'vue'
48 const instance = getCurrentInstance()
49 if (instance) {
50     instance.appContext.config.globalProperties.$router = router
51 }

```

2. utils/request.js (请求封装, 含拦截器)

```

1  import { useUserStore } from '@store/modules/user.js'
2
3  // 基础请求地址 (根据实际项目修改)
4  const baseUrl = 'https://api.example.com'
5
6  // 加载提示 (可自定义)
7  const showLoading = () => {
8      uni.showLoading({
9          title: '加载中...',
10         mask: true
11     })
12 }
13
14 // 关闭加载提示
15 const hideLoading = () => {
16     uni.hideLoading()
17 }
18
19 /**
20  * 请求封装 (含请求/响应拦截)
21  * @param {Object} options - 请求配置
22  * @param {String} options.url - 请求地址 (不含baseUrl)
23  * @param {String} [options.method=GET] - 请求方法
24  * @param {Object} [options.data={}] - 请求参数
25  * @param {Boolean} [options.showLoad=true] - 是否显示加载提示
26  * @returns {Promise} - 请求结果
27  */
28 export function request(options) {
29     const { url, method = 'GET', data = {}, showLoad = true } = options

```

```
30     const userStore = useUserStore()
31
32     // 请求拦截：显示加载、添加token
33     showLoad && showLoading()
34     const header = {
35       'Content-Type': 'application/json',
36       // 从Pinia中获取token，添加到请求头（登录后生效）
37       'Authorization': `Bearer ${userStore.token || ''}`
38     }
39
40     return new Promise((resolve, reject) => {
41       uni.request({
42         url: baseUrl + url,
43         method,
44         data,
45         header,
46         // 响应拦截
47         success: (res) => {
48           hideLoading()
49           const { code, data, message } = res.data
50
51           // 统一错误处理（根据后端接口规范修改）
52           if (code === 200) {
53             // 成功：返回响应数据
54             resolve(data)
55           } else if (code === 401) {
56             // token过期/未登录：跳转登录页，清除token
57             userStore.clearToken()
58             uni.showToast({ title: '请先登录', icon: 'none' })
59             setTimeout(() => {
60               uni.reLaunch({ url: '/pages/login/login' }) // 可添加登录页
61             }, 1000)
62             reject('未登录或token过期')
63           } else {
64             // 其他错误：提示错误信息
65             uni.showToast({ title: message || '请求失败', icon: 'none' })
66             reject(message || '请求失败')
67           }
68         },
69         // 请求失败（网络错误等）
70         fail: (err) => {
71           hideLoading()
72           uni.showToast({ title: '网络异常，请稍后再试', icon: 'none' })
73           reject(err)
74         }
75       })
76     })
```



```
77 }
78
79 // 简化GET请求
80 export const get = (url, data = {}, showLoad = true) => {
81   return request({ url, data, method: 'GET', showLoad })
82 }
83
84 // 简化POST请求
85 export const post = (url, data = {}, showLoad = true) => {
86   return request({ url, data, method: 'POST', showLoad })
87 }
```

四、Pinia 全局状态管理

1. store/index.js (Pinia入口, 统一注册模块)

```
1 import { createPinia } from 'pinia'
2
3 // 创建Pinia实例
4 const pinia = createPinia()
5
6 // 导出Pinia, 在main.js中注册
7 export default pinia
8
9 // 导出所有模块 (方便页面引入)
10 export * from './modules/user.js'
```

2. store/modules/user.js (用户模块, 示例)

```
1 import { defineStore } from 'pinia'
2
3 // 定义用户状态模块 (useUserStore为自定义名称, 需唯一)
4 export const useUserStore = defineStore('user', {
5   // 状态 (类似Vue2的data)
6   state: () => ({
7     token: uni.getStorageSync('token') || '', // token, 持久化存储
8     userInfo: uni.getStorageSync('userInfo') || {} // 用户信息
9   }),
10
11   // 计算属性 (类似Vue2的computed)
12   getters: {
```

```

13      // 判断是否登录
14      isLogin: (state) => !!state.token
15    },
16
17    // 方法（类似Vue2的methods，可修改状态）
18    actions: {
19      // 保存token（并持久化到本地存储）
20      setToken(token) {
21        this.token = token
22        uni.setStorageSync('token', token)
23      },
24
25      // 保存用户信息
26      setUserInfo(userInfo) {
27        this.userInfo = userInfo
28        uni.setStorageSync('userInfo', userInfo)
29      },
30
31      // 清除token和用户信息（退出登录）
32      clearToken() {
33        this.token = ''
34        this.userInfo = {}
35        uni.removeStorageSync('token')
36        uni.removeStorageSync('userInfo')
37      }
38    }
39  })

```

五、页面与组件示例

1. 首页 pages/index/index.vue

```

1  <template>
2    <view class="index-container">
3      <text class="title">{{ msg }}</text>
4
5      <!-- 测试路由跳转（带参数） -->
6      <button class="btn" @click="goToDetail" type="primary">跳转到详情页
7    </button>
8
9      <!-- 测试tabBar跳转 -->
10     <button class="btn" @click="goToMine" type="default">跳转到我的</button>
11
12     <!-- 测试请求（需替换实际接口） -->

```

```
12     <button class="btn" @click="getTestData" type="success">测试请求</button>
13
14     <!-- 测试组件 -->
15     <Hello />
16
17     <!-- 测试Pinia状态 -->
18     <view class="pinia-test" v-if="userStore.isLogin">
19         欢迎您: {{ userStore.userInfo.nickname }}
20     </view>
21 </view>
22 </template>
23
24 <script setup>
25 import { ref, onMounted } from 'vue'
26 // 引入组件
27 import Hello from '@/components/Hello.vue'
28 // 引入路由
29 import { router } from '@/utils/router.js'
30 // 引入请求
31 import { get } from '@/utils/request.js'
32 // 引入Pinia用户状态
33 import { useUserStore } from '@/store/modules/user.js'
34
35 // 响应式数据
36 const msg = ref('uniApp Vue3 完整模板 - 首页')
37 const userStore = useUserStore()
38
39 // 生命周期
40 onMounted(() => {
41     console.log('首页挂载完成')
42     // 示例：获取本地存储的用户信息
43     console.log('当前用户: ', userStore.userInfo)
44 })
45
46 // 路由跳转：跳转到详情页（带参数）
47 const goToDetail = () => {
48     router.push('/pages/detail/index', { id: 1, name: '测试详情' })
49 }
50
51 // 路由跳转：跳转到tabBar页面（我的）
52 const goToMine = () => {
53     router.switchTab('/pages/mine/mine')
54 }
55
56 // 测试请求（替换为自己的接口）
57 const getTestData = async () => {
58     try {
```

```

59     const data = await get('/api/test') // 接口地址需和baseUrl拼接
60     console.log('请求成功: ', data)
61     uni.showToast({ title: '请求成功', icon: 'success' })
62   } catch (err) {
63     console.log('请求失败: ', err)
64   }
65 }
66 </script>
67
68 <style scoped>
69   .index-container {
70     padding: 30rpx;
71   }
72   .title {
73     font-size: 36rpx;
74     font-weight: bold;
75     margin-bottom: 30rpx;
76     display: block;
77     text-align: center;
78   }
79   .btn {
80     margin-bottom: 20rpx;
81     width: 100%;
82   }
83   .pinia-test {
84     margin-top: 30rpx;
85     text-align: center;
86     color: #409eff;
87     font-size: 30rpx;
88   }
89 </style>

```

2. 我的页面 pages/mine/mine.vue

```

1  <template>
2    <view class="mine-container">
3      <view class="user-info">
4        <image src="/static/avatar.png" class="avatar" mode="widthFix"></image>
5        <view class="user-name">
6          {{ userStore.isLogin ? userStore.userInfo.nickname : '请登录' }}
7        </view>
8      </view>
9
10     <button
11       class="login-btn"

```

```
12     @click="handleLogin"
13     :type="userStore.isLogin ? 'default' : 'primary'"
14   >
15     {{ userStore.isLogin ? '退出登录' : '去登录' }}
16   </button>
17 </view>
18 </template>
19
20 <script setup>
21 import { useUserStore } from '@store/modules/user.js'
22 import { router } from '@utils/router.js'
23
24 const userStore = useUserStore()
25
26 // 登录/退出登录
27 const handleLogin = () => {
28   if (userStore.isLogin) {
29     // 退出登录：清除状态
30     userStore.clearToken()
31     uni.showToast({ title: '退出成功', icon: 'success' })
32   } else {
33     // 去登录（可替换为实际登录页路径）
34     router.push('/pages/login/login')
35   }
36 }
37 </script>
38
39 <style scoped>
40 .mine-container {
41   padding: 30rpx;
42 }
43 .user-info {
44   display: flex;
45   align-items: center;
46   margin-bottom: 40rpx;
47   flex-direction: column;
48 }
49 .avatar {
50   width: 160rpx;
51   height: 160rpx;
52   border-radius: 50%;
53   margin-bottom: 20rpx;
54 }
55 .user-name {
56   font-size: 32rpx;
57   font-weight: 500;
58 }
```

```
59 .login-btn {
60   width: 100%;
61 }
62 </style>
```

3. 详情页 pages/detail/index.vue

```
1  <template>
2    <view class="detail-container">
3      <text class="title">详情页</text>
4      <view class="params">
5        <text>接收的参数: </text>
6        <text>id: {{ id }}, 名称: {{ name }}</text>
7      </view>
8      <button class="back-btn" @click="goBack" type="default">返回上一页</button>
9    </view>
10 </template>
11
12 <script setup>
13 import { ref, onLoad } from 'vue'
14 import { router } from '@/utils/router.js'
15
16 // 接收路由参数
17 const id = ref('')
18 const name = ref('')
19
20 // 页面加载时获取参数
21 onLoad((options) => {
22   id.value = options.id
23   name.value = options.name
24   console.log('接收的参数: ', options)
25 })
26
27 // 返回上一页
28 const goBack = () => {
29   router.back()
30 }
31 </script>
32
33 <style scoped>
34 .detail-container {
35   padding: 30rpx;
36 }
37 .title {
38   font-size: 36rpx;
```

```
39     font-weight: bold;
40     margin-bottom: 30rpx;
41     display: block;
42     text-align: center;
43 }
44 .params {
45     margin: 30rpx 0;
46     font-size: 30rpx;
47     line-height: 50rpx;
48 }
49 .back-btn {
50     width: 100%;
51     margin-top: 40rpx;
52 }
53 </style>
```

4. 组件 components/Hello.vue

```
1  <template>
2    <view class="hello-component">
3      <text class="hello-text">这是Vue3组件示例</text>
4      <text class="tips">可直接复用，支持setup语法</text>
5    </view>
6  </template>
7
8  <script setup>
9    // 组件内部可使用Vue3所有语法
10   import { ref } from 'vue'
11   const msg = ref('组件内部响应式数据')
12   console.log('组件加载: ', msg.value)
13 </script>
14
15 <style scoped>
16   .hello-component {
17     margin-top: 30rpx;
18     padding: 20rpx;
19     background-color: #e6f7ff;
20     border-radius: 10rpx;
21     text-align: center;
22   }
23   .hello-text {
24     font-size: 32rpx;
25     color: #409eff;
26     font-weight: 500;
27     display: block;
```

```
28     margin-bottom: 10rpx;
29   }
30   .tips {
31     font-size: 26rpx;
32     color: #666;
33   }
34   </style>
```

六、使用说明（必看）

1. 复制上述所有文件，按照「项目完整结构」创建对应文件夹和文件，确保路径一致；
2. 打开终端，进入项目根目录，执行 `npm install` 安装依赖；
3. 修改配置：
 - manifest.json：替换 appid（uniApp appid、微信小程序 appid）；
 - utils/request.js：修改 baseUrl 为自己的项目接口地址；
 - tabBar 图标：替换 static 目录下的图标，或修改 pages.json 中的图标路径；
4. 运行项目（终端执行对应命令）：
 - H5： `npm run dev:h5` ；
 - 微信小程序： `npm run dev:mp-weixin` ，然后在微信开发者工具中导入 dist/dev/mp-weixin 目录；
 - App： `npm run dev:app` ，连接真机或模拟器运行；
5. 打包发布：执行对应 build 命令（如 `npm run build:mp-weixin` ），然后提交审核。

七、补充说明

- 本模板适配 uniApp Vue3 最新版本，使用 setup 语法糖，简洁高效；
- 路由、请求、Pinia 均已封装，可直接使用，无需重复开发；
- 可根据实际需求扩展：添加登录页、更多Pinia模块、自定义组件等；
- 多端适配：通过条件编译（ `// #ifdef MP-WEIXIN` 等）处理不同平台差异，模板已兼容基础多端需求。

（注：文档部分内容可能由 AI 生成）